
UNIVERSITÉ DU SINE SALOUM EL HADJI IBRAHIMA NIASS**UFR des Sciences Fondamentales et de l'Ingénierie****Département de Mathématiques-Informatique**

Travaux Dirigés

Algorithmique et Programmation II

Thème : Les Tableaux

Enseignant : Dr B. DIOP**Partie I : Tableaux 1D – Parcours et recherche****Exercice 1. Recherche des deux plus grands éléments**

Saisir 10 entiers dans un tableau et déterminer, **en un seul parcours**, le plus grand et le deuxième plus grand élément.

Contrainte : Vous ne devez parcourir le tableau qu'une seule fois (complexité $O(n)$).

Exercice 2. Vérification de l'ordre croissant

Écrire une fonction qui prend un tableau en argument et retourne **vrai** si tous ses éléments sont rangés en ordre croissant, **faux** sinon.

Prototype suggéré :

```
int estTrieCroissant(int tab[], int taille);
```

Exercice 3. Comparaison de tableaux

Écrire une fonction :

```
int compare(const float t1[], const float t2[], int size);
```

Cette fonction reçoit deux tableaux en paramètres et retourne le nombre d'éléments ayant **le même indice et la même valeur** dans les deux tableaux.

Test : Tester la fonction avec les tableaux suivants :

```
float notes1[NB_NOTES] = {10.2, 12.5, 18.5, 9.8, 13.2, 12.1};  
float notes2[NB_NOTES] = {15.7, 12.5, 15.3, 9.8, 10.8, 12.1};
```

Résultat attendu : La fonction doit retourner 3 (indices 1, 3 et 5).

Exercice 4. Égalité de deux tableaux

Écrire une fonction `egal()` qui reçoit deux tableaux en paramètres. Cette fonction retourne :

- 1 si les deux tableaux sont strictement identiques ;
- 0 sinon.

Exercice 5. Vérification de sous-ensemble

Écrire un programme en C pour vérifier si un tableau est un sous-ensemble d'un autre tableau (tous les éléments du second tableau sont présents dans le premier).

Exemple :

```
Premier tableau : 3 8 7 12 6 9 1 0 5
Deuxième tableau : 1 3 7 6 0
Résultat : Le deuxième tableau est un sous-ensemble du premier.
```

Exercice 6. Nombres successifs

Écrire un programme en C qui permet d'entrer des nombres entiers et de vérifier si ces nombres sont successifs (consécutifs) ou non.

Exemples :

```
Entrée : 0 1 3 4 5 6 7 8 9
Sortie : Les nombres ne sont pas successifs

Entrée : 11 12 13
Sortie : Les nombres sont successifs
```

Exercice 7. Recherche dichotomique

On cherche un élément x dans un tableau trié. Implémenter la **recherche dichotomique** :

- À chaque itération, on compare x avec l'élément du milieu de la zone de recherche.
- Si x est plus petit, on restreint la recherche à la moitié gauche ; sinon, à la moitié droite.
- On répète jusqu'à trouver l'élément ou déterminer qu'il est absent.

Question : Quelle est la complexité de cet algorithme ?

Partie II : Tableaux 1D – Statistiques et comptage**Exercice 8. Éléments uniques**

Écrire un programme en C pour afficher tous les éléments uniques d'un tableau (éléments qui n'apparaissent qu'une seule fois).

Exemple :

```
Entrer le nombre d'éléments : 4
Entrer les 4 éléments :
tab[0]: 5
tab[1]: 7
tab[2]: 7
tab[3]: 1
Les éléments uniques sont : 5 1
```

Exercice 9. Fréquence des éléments

Écrire un programme en C pour compter la fréquence de chaque élément dans un tableau.

Exemple :

```
Entrer le nombre d'éléments : 3
Entrer les 3 éléments :
tab[0]: 8
tab[1]: 8
tab[2]: 9
Fréquence des éléments :
8 apparaît 2 fois
9 apparaît 1 fois
```

Exercice 10. Élément majoritaire

Écrire un programme en C pour trouver l'élément majoritaire d'un tableau. Un élément est dit **majoritaire** s'il apparaît plus de $n/2$ fois dans un tableau de taille n .

Exemple :

```
Tableau donné : 4 1 2 4 4 5 4 6 8 4
L'élément majoritaire est : 4
```

Exercice 11. Différence pairs et impairs

Écrire un programme en C qui permet la saisie de 5 nombres dans un tableau et qui calcule la différence entre la somme des nombres pairs et la somme des nombres impairs.

Exemple :

```
Entrée : 3 5 2 4 6
Calcul : (2+4+6) - (3+5) = 12 - 8 = 4
```

Exercice 12. Plus petit positif manquant

Écrire un programme en C pour trouver le plus petit nombre entier positif manquant dans un tableau non trié.

Exemple :

```
Tableau donné : 3 1 2 -5 5 -23 10 6 8 -7
Le plus petit nombre positif manquant est : 4
```

Explication : Les entiers positifs présents sont $\{1, 2, 3, 5, 6, 8, 10\}$. Le premier manquant est 4.

Partie III : Tableaux 1D – Modifications et réorganisation

Exercice 13. Suppression des doublons

Écrire un programme qui lit un tableau d'entiers, puis supprime toutes les occurrences multiples d'un élément pour ne conserver que des valeurs uniques. Le tableau résultant doit être compacté (les éléments sont décalés vers la gauche pour combler les trous).

Exemple :

- Entrée : $[3, 1, 4, 1, 5, 9, 2, 6, 5, 3]$
- Sortie : $[3, 1, 4, 5, 9, 2, 6]$

Exercice 14. Insertion dans un tableau

On dispose d'un tableau de n éléments, avec une case supplémentaire disponible à la fin. Écrire un algorithme qui insère un nouvel élément à une position p donnée, en décalant les éléments suivants vers la droite.

Exemple :

- Tableau initial : [2, 5, 8, 12, _]
- Insertion de 7 à la position 2
- Tableau final : [2, 5, 7, 8, 12]

Exercice 15. Insertion dans un tableau trié

Écrire un programme en C pour insérer une valeur dans un tableau trié, de manière à ce que le tableau reste trié après l'insertion.

Exemple :

```
Tableau initial : 2 4 8
Valeur à insérer : 7
Tableau après insertion : 2 4 7 8
```

Exercice 16. Suppression à une position donnée

Écrire un programme en C pour supprimer un élément d'un tableau à une position donnée, puis compacter le tableau.

Exemple :

```
Tableau initial : 2 4 8
Position à supprimer : 2
Tableau après suppression : 2 8
```

Exercice 17. Déplacement des zéros

Écrire un programme en C qui déplace tous les zéros vers la fin du tableau, tout en maintenant l'ordre relatif des autres éléments.

Exemple :

```
Entrée : 8 0 6 0 1 6 0 0 2 3
Sortie : 8 6 1 6 2 3 0 0 0 0
```

Exercice 18. Conversion décimal-binaire

Demander à l'utilisateur un entier en base décimale, stocker sa représentation binaire dans un tableau, puis afficher ce nombre en binaire.

Exemple :

- Entrée : 13
- Sortie : 1101

Partie IV : Tableaux 1D – Tri et fusion

Exercice 19. Tri à bulles

Écrire une fonction `tri()` qui reçoit un tableau et sa taille en paramètres. La fonction trie le tableau dans l'ordre croissant.

Prototype :

```
void tri(int tab[], int size);
```

Indication : Vous pouvez vous inspirer du tri à bulles.

Exemple :

```
Tableau initial : 12 54 0 18 8 98 5 16 58 4 75 5
Tableau trié    : 0 4 5 5 8 12 16 18 54 58 75 98
```

Exercice 20. Fusion de deux tableaux triés

On considère deux tableaux triés T_1 et T_2 de tailles respectives n_1 et n_2 . Écrire un algorithme qui fusionne ces deux tableaux en un seul tableau trié T_3 de taille $n_1 + n_2$.

Exemple :

- $T_1 = [1, 4, 7, 9]$
- $T_2 = [2, 3, 8, 10, 12]$
- $T_3 = [1, 2, 3, 4, 7, 8, 9, 10, 12]$

Exercice 21. Méli-mélo avec deux tableaux

Proposer un algorithme qui transfère les n éléments d'un tableau T_1 vers un tableau T_2 selon le principe suivant :

- Les éléments de rang **impair** de T_1 sont rangés dans T_2 en ordre inverse, en commençant par la fin de T_2 .
- Les éléments de rang **pair** de T_1 sont rangés dans T_2 dans le même ordre, au début de T_2 .

Donner la traduction en C de cet algorithme.

Partie V : Tableaux 2D – Matrices

Exercice 22. Détection des points creux

Une matrice M de dimension $m \times n$ est donnée. Un élément $M[i][j]$ est appelé **point creux** s'il est strictement inférieur à tous ses voisins immédiats (haut, bas, gauche, droite).

Écrire un programme qui identifie et affiche tous les points creux de la matrice ainsi que leurs indices.

Remarque : Les éléments sur les bords ont moins de 4 voisins.

Exercice 23. Parcours en accordéon

Écrire une fonction qui affiche les éléments d'une matrice selon un parcours en **accordéon** (ou zigzag) :

- La première ligne est parcourue de gauche à droite.
- La deuxième ligne est parcourue de droite à gauche.
- On alterne ainsi de suite.

Exemple : Pour la matrice

$$\begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 \end{pmatrix}$$

L’affichage doit être : 0, 1, 2, 3, 7, 6, 5, 4, 12, 13, 14, 15, 19, 18, 17, 16.

Exercice 24. Ligne avec le maximum de 1

Écrire un programme en C pour trouver l’indice de la ligne contenant le plus grand nombre de 1 dans une matrice binaire.

Exemple :

Le tableau 2D donné est :

0 0 1 1

0 0 1 1

1 1 1 1

0 0 0 0

L’index de la ligne avec le maximum de 1 est : 2

Exercice 25. Sommes des diagonales

Écrire un programme en C pour calculer les sommes des diagonales principale (gauche) et secondaire (droite) d’une matrice carrée.

Exemple :

La matrice est :

1 2

3 4

Somme diagonale principale (gauche) : 1 + 4 = 5

Somme diagonale secondaire (droite) : 2 + 3 = 5

Exercice 26. Transformation en matrice triangulaire inférieure

On dispose d’une matrice $N \times N$ composée uniquement de 0 et de 1. La seule opération autorisée est l’échange de deux lignes adjacentes.

Une matrice est dite **triangulaire inférieure** si, pour chaque ligne i (avec $1 \leq i \leq N$), tous les éléments en position (i, j) avec $j > i$ sont égaux à 0.

Question : Déterminer le nombre minimum d’échanges de lignes adjacentes nécessaires pour obtenir une matrice triangulaire inférieure, ou indiquer que c’est impossible.

Partie VI : Tableaux parallèles et gestion de données

Exercice 27. Gestion de notes

Créer deux tableaux parallèles :

- Un tableau de chaînes de caractères pour les noms des étudiants : `char noms[50][20]`
- Un tableau de réels pour leurs notes : `float notes[50]`

Écrire un programme qui :

1. Saisit les données de n étudiants.

2. Calcule et affiche la moyenne de la classe.
3. Affiche le nom du meilleur étudiant.
4. Trie et affiche les étudiants par ordre de mérite (note décroissante).

Partie VII : Problèmes algorithmiques avancés

Exercice 28. Minimisation du produit scalaire

On considère deux vecteurs de dimension n :

$$\vec{v}_1 = (x_1, x_2, \dots, x_n) \quad \text{et} \quad \vec{v}_2 = (y_1, y_2, \dots, y_n)$$

Le produit scalaire est défini par :

$$\vec{v}_1 \cdot \vec{v}_2 = \sum_{i=1}^n x_i \cdot y_i$$

On peut réordonner librement les composantes de chaque vecteur.

Questions :

1. Trouver deux permutations σ et τ des indices $\{1, 2, \dots, n\}$ telles que le produit scalaire soit **minimal**.
2. Calculer cette valeur minimale.

Exemple : $\vec{v}_1 = (1, 3, -5)$ et $\vec{v}_2 = (-2, 4, 1)$.

Produit scalaire minimal : $(-5) \times 4 + 1 \times 1 + 3 \times (-2) = -25$

Exercice 29. Sous-séquences strictement croissantes

Soit une séquence $S = (a_1, a_2, \dots, a_n)$ de n entiers.

Une **sous-séquence strictement croissante** de S est une suite $(a_{i_1}, a_{i_2}, \dots, a_{i_k})$ telle que :

- $i_1 < i_2 < \dots < i_k$ (indices croissants)
- $a_{i_1} < a_{i_2} < \dots < a_{i_k}$ (valeurs strictement croissantes)
- $k \geq 1$ (au moins un élément)

Attention : Deux sous-séquences sont distinctes si elles diffèrent par au moins une position, même si les valeurs sont identiques.

Question : Calculer le nombre total de sous-séquences strictement croissantes, modulo un nombre premier P .

Contraintes : $1 \leq n \leq 500\,000$

Exercice 30. Groupement par facteurs premiers communs

On considère une suite d'entiers consécutifs de A à B (inclus). Initialement, chaque entier forme son propre ensemble.

Pour chaque paire d'entiers dans $[A, B]$: si ces deux entiers partagent un facteur premier $\geq P$, on fusionne leurs ensembles.

Question : Déterminer le nombre d'ensembles distincts à la fin du processus.

Exercice 31. Soudoyer les prisonniers

Dans un royaume, P cellules sont disposées en ligne (1 à P). Les prisonniers adjacents communiquent entre eux.

Lorsqu'un prisonnier est libéré de la cellule A , l'information se propage jusqu'aux extrémités ou une cellule vide. Chaque prisonnier informé devient furieux sauf s'il reçoit une pièce d'or.

On doit libérer Q prisonniers (un par jour), dans l'ordre de notre choix.

Question : Déterminer l'ordre de libération qui minimise le nombre total de pièces d'or.

Exercice 32. Centre de masse d'un essaim

On étudie N lucioles se déplaçant en ligne droite dans l'espace 3D. Chaque luciole i a :

- Position initiale : (x_i, y_i, z_i)
- Vitesse : $(v_{x_i}, v_{y_i}, v_{z_i})$

Le centre de masse au temps t est : $M(t) = \frac{1}{N} \sum_{i=1}^N P_i(t)$

Questions : Trouver $d_{\min}$ (distance minimale à l'origine) et $t_{\min}$ (instant correspondant).

Exercice 33. Couverture par carrés

On dispose d'une grille de carreaux bleus (#) et blancs (.). On veut couvrir tous les carreaux bleus par des **carreaux rouges** 2×2 , sans chevauchement ni débordement.

Question : Déterminer si c'est possible.

Exercice 34. Harmonie parfaite

Un orchestre joue des fréquences f_i (entiers). Deux sons sont en harmonie si l'un divise l'autre.

Question : Trouver une fréquence $f \in [L, H]$ en harmonie avec toutes les autres, ou indiquer que c'est impossible.

Exercice 35. Plan de voyage spatial

N planètes sont alignées. La Terre est à $X_1 = 0$. Vous avez F unités de carburant.

Coût du trajet $i \rightarrow j$: $|X_i - X_j|$ unités.

Question : Visiter chaque planète une fois, revenir sur Terre, et **maximiser** la consommation (sans dépasser F).

Exercice 36. Chemins du Yin et du Yang

Une grille $N \times M$ contient des cellules noires (Yin) et blanches (Yang). La grille est valide si chaque couleur forme un chemin (connexe, 2 extrémités, autres cellules avec 2 voisins).

Question : Compter le nombre de grilles valides.

Exercice 37. Simulation de bactéries

Des bactéries vivent sur une grille infinie. À chaque seconde :

1. Une bactérie sans voisin au nord ni à l'ouest meurt.
2. Une cellule vide avec voisins nord **et** ouest occupés donne naissance.

Question : Déterminer quand toutes les bactéries auront disparu.

Exercice 38. Intervalles intéressants

Un palindrome se lit identiquement à l'endroit et à l'envers (ex: 363, 4884).

Un intervalle $[L, R]$ est **intéressant** s'il contient un nombre **pair** de palindromes.

Question : Déterminer si $[L, R]$ est intéressant.

Bon courage !